

Convolutional Neural Networks

Yu Kong

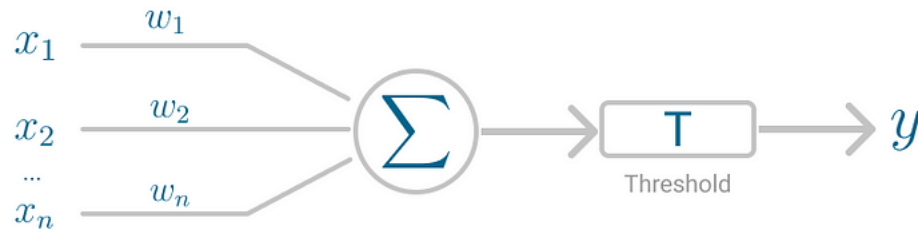
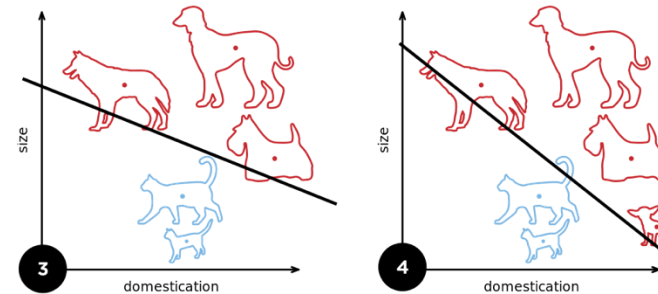
Department of Computer Science and Engineering

Michigan State University

Multi-layer perceptron

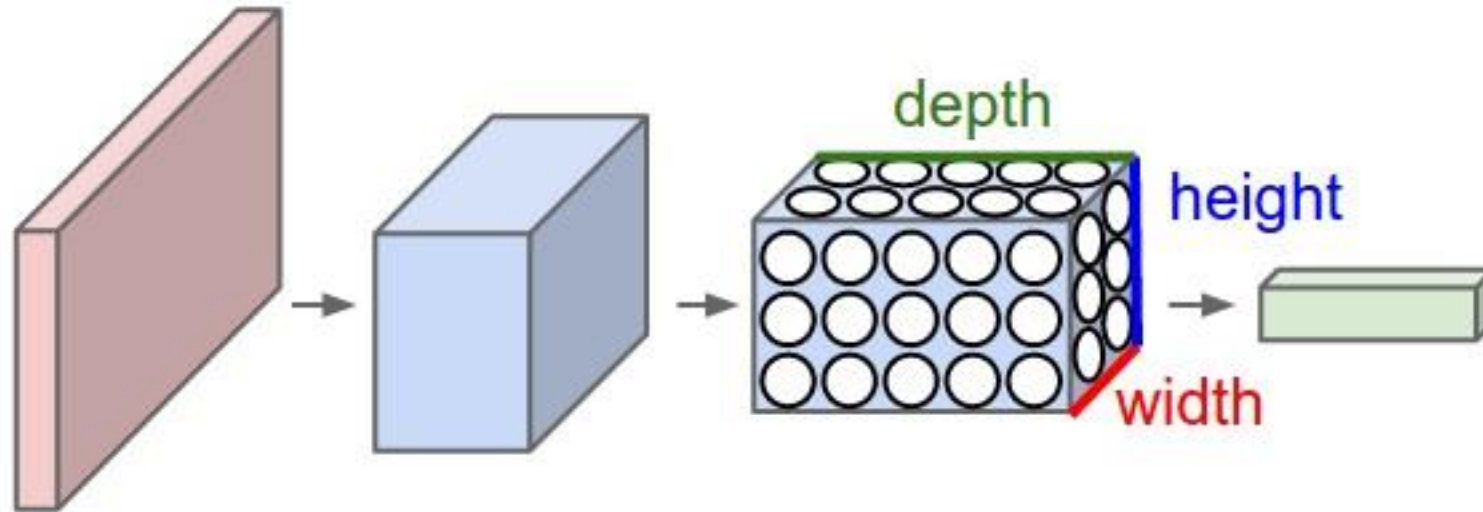
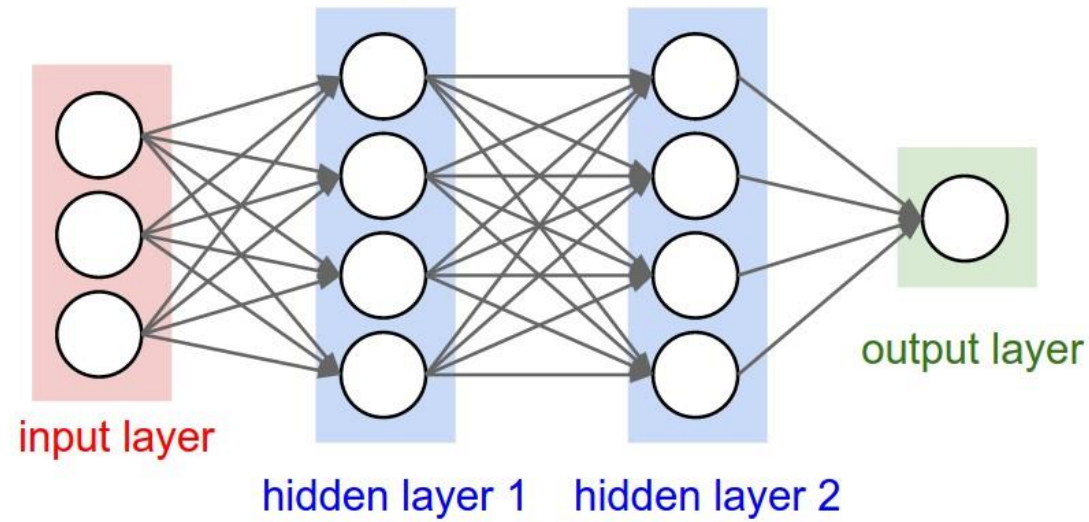
- CNNs: bio-inspired **multi-layer perceptron** (MLP)
- Perceptron: an algorithm for supervised learning of binary classifiers

$$f(x) = \begin{cases} 1 & \text{if } w \cdot x + b > 0 \\ 0 & \text{otherwise} \end{cases}$$



$$y = \begin{cases} 1, & \text{if } \overbrace{\sum_i w_i x_i}^{\text{weighted sum}} - \underbrace{T}_{\text{threshold}} > 0 \\ 0, & \text{otherwise} \end{cases}$$

CNN



A ConvNet arranges its neurons in three dimensions (width, height, depth), as visualized in one of the layers. Every layer of a ConvNet transforms the 3D input volume to a 3D output volume of neuron activations.

Conventional approaches

- Edge detectors
 - Canny detectors
 - Sobel filter
 - ...
- Feature engineering.
- We don't know which one is useful. Can we use more filters together?
 - Convolutional nets



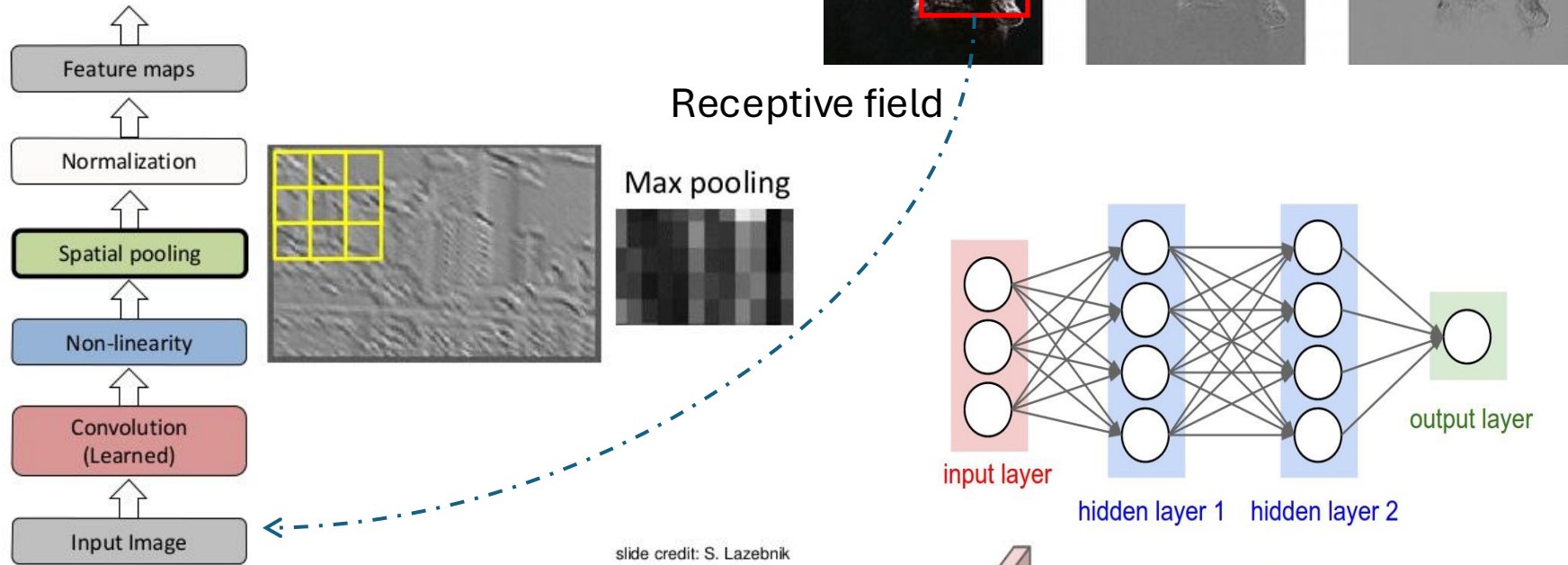
Weights are learned in CNN

$$\begin{pmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{pmatrix} \longrightarrow \begin{pmatrix} w_1 & w_2 & w_3 \\ w_4 & w_5 & w_6 \\ w_7 & w_8 & w_9 \end{pmatrix}$$

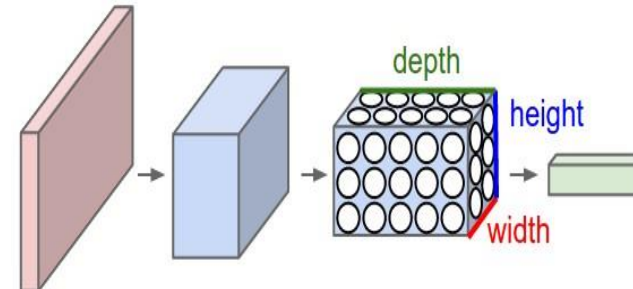
CNN

- Variants of MLP

Convolutional Neural Networks

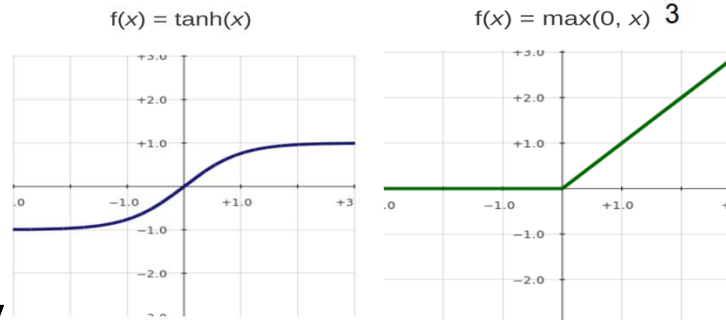
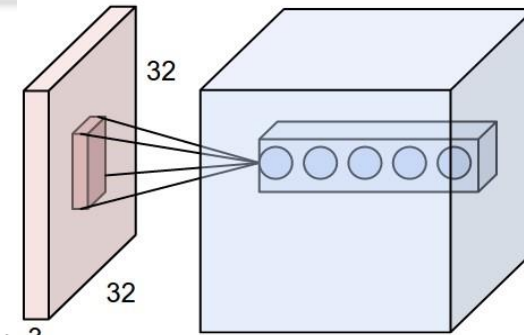


$$5 \times 5 \times 3 = \mathbf{75} \text{ weights}$$

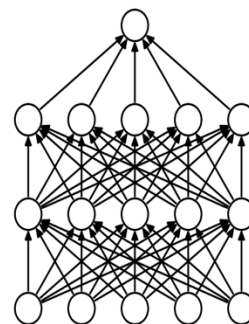
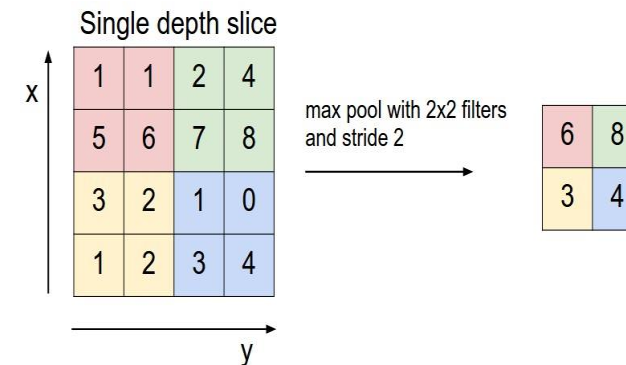


Layers

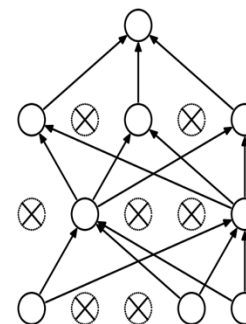
- **Convolutional layer**
 - Intertwine two sources
 - Capture salient information.
- **ReLU layer**
 - Normalization
- **Pooling layer**
 - Dimensionality reduction.
- **Dropout method**
- **Loss layer**



$$f(x) = \max(0, x)$$
$$f(x) = \tanh(x)$$
$$f(x) = (1 + e^{-x})^{-1}$$

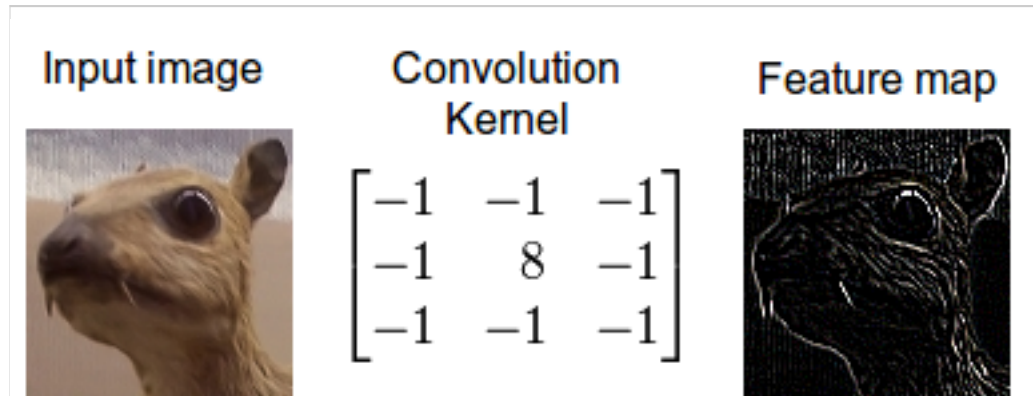


(a) Standard Neural Net



(b) After applying dropout.

Convolutional Layer



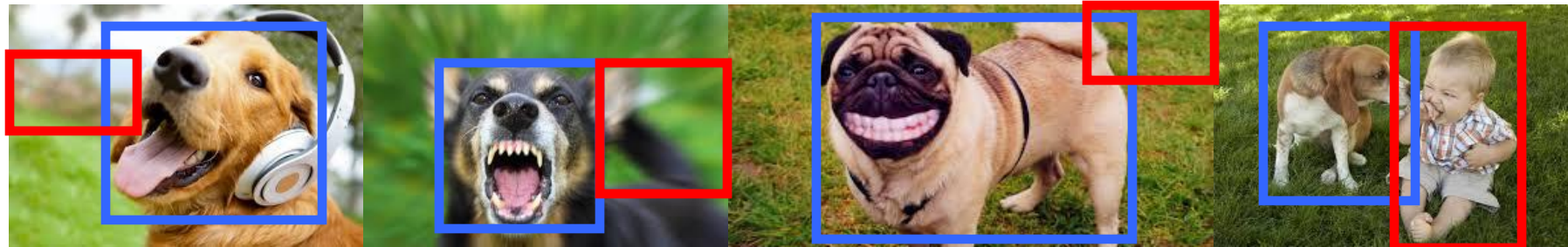
(<https://timdettmers.files.wordpress.com/2015/03/convolution.png>)

Convolution of an image with an edge detector convolution kernel. Sources: 1 (<http://en.wikipedia.org/wiki/File:Vd-Orig.png>) 2 (<http://en.wikipedia.org/wiki/File:Vd-Edge3.png>)

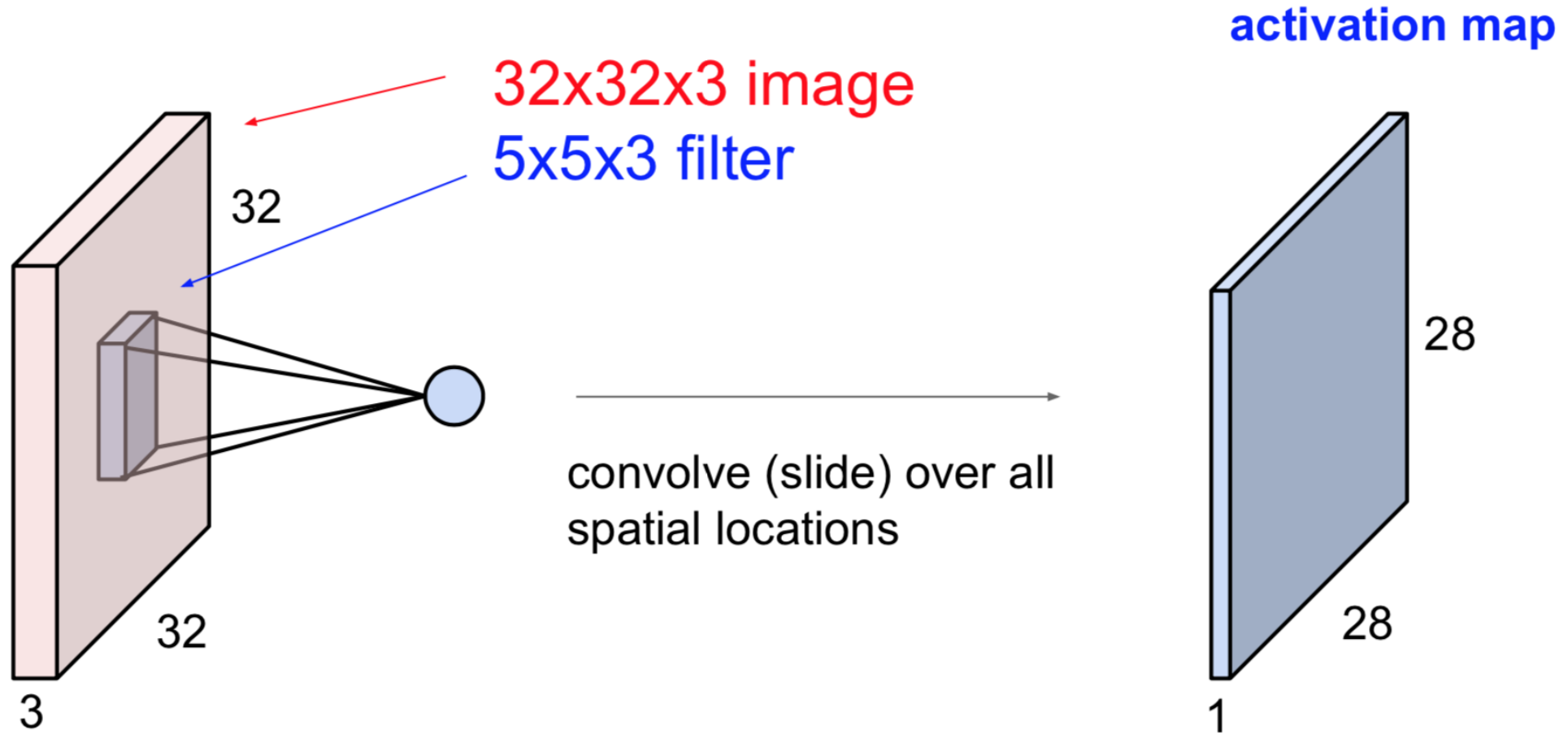
Feature maps:
Red, Green, Blue



Images: lots of distracting information.
How can we filter out this noise information?



Convolution Layer



Convolution for 1 channel

Kernel/Filter, K =

1	0	1
0	1	0
1	0	1

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature

Exercise

Image

1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0

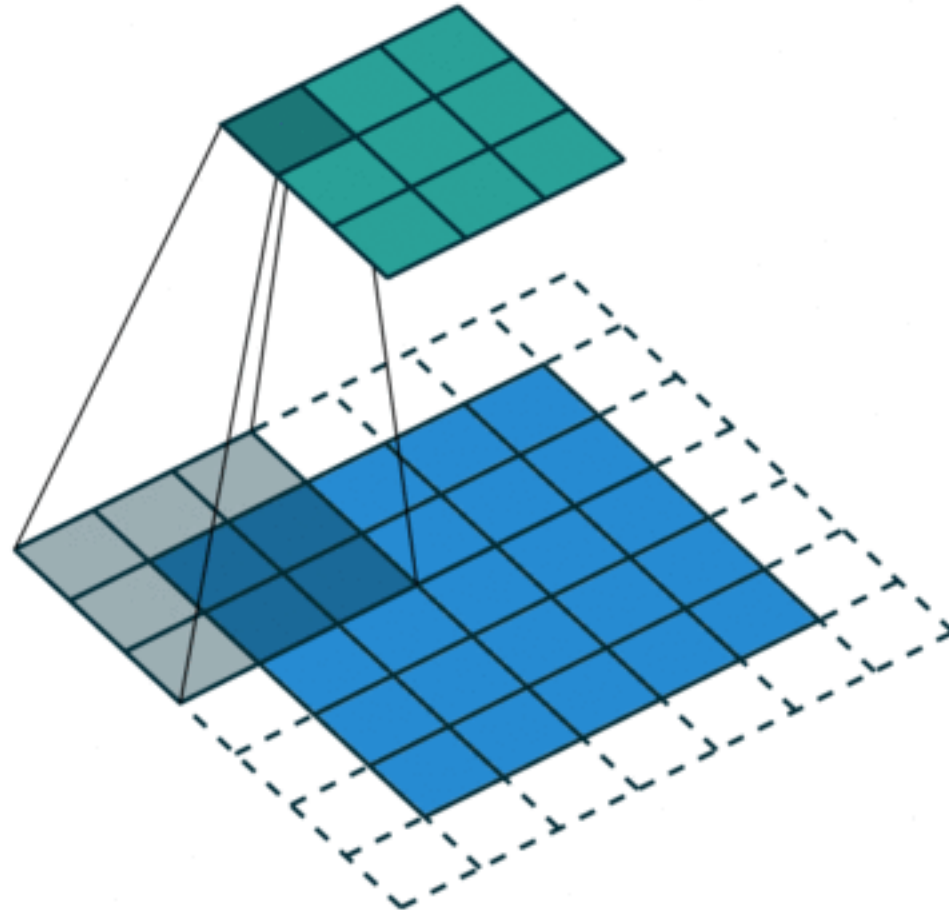
Filter

1	0	-1
1	0	-1
1	0	-1

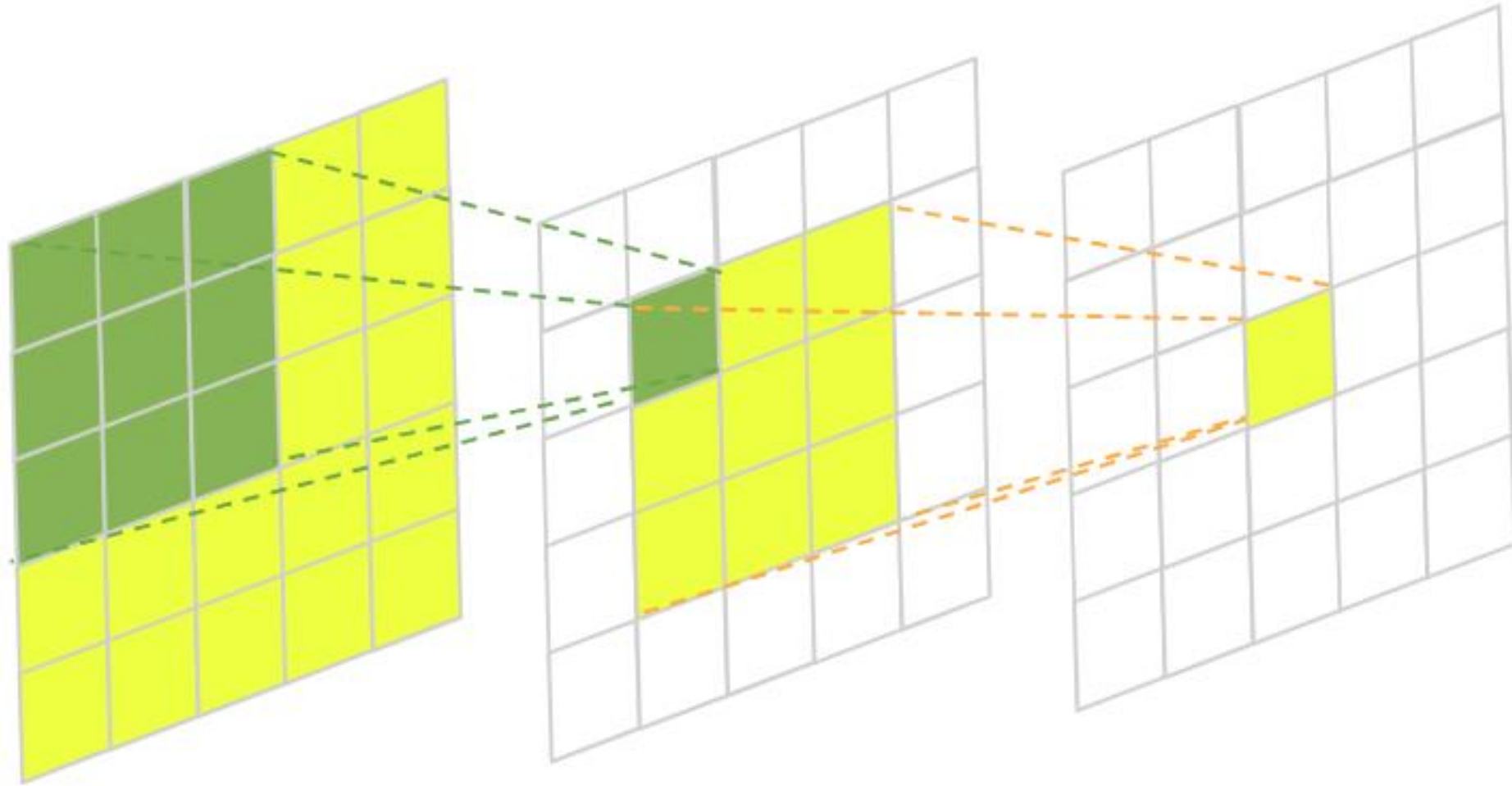
Filter

1	1	1
0	0	0
-1	-1	-1

Stride



Receptive field



Convolution for 3 channels

0	0	0	0	0	0	...
0	156	155	156	158	158	...
0	153	154	157	159	159	...
0	149	151	155	158	159	...
0	146	146	149	153	158	...
0	145	143	143	148	158	...
...	...	...	...	...	...	...

Input Channel #1 (Red)

0	0	0	0	0	0	...
0	167	166	167	169	169	...
0	164	165	168	170	170	...
0	160	162	166	169	170	...
0	156	156	159	163	168	...
0	155	153	153	158	168	...
...	...	...	...	...	...	...

Input Channel #2 (Green)

0	0	0	0	0	0	...
0	163	162	163	165	165	...
0	160	161	164	166	166	...
0	156	158	162	165	166	...
0	155	155	158	162	167	...
0	154	152	152	157	167	...
...	...	...	...	...	...	...

Input Channel #3 (Blue)

-1	-1	1
0	1	-1
0	1	1

Kernel Channel #1



308

1	0	0
1	-1	-1
1	0	-1

Kernel Channel #2



-498

0	1	1
0	1	0
1	-1	1

Kernel Channel #3



164

+ 1 = -25

Bias = 1

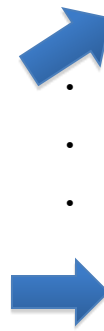
Output

-25				...
				...
				...
				...
...	...	...	...	...

Convolution as feature extraction

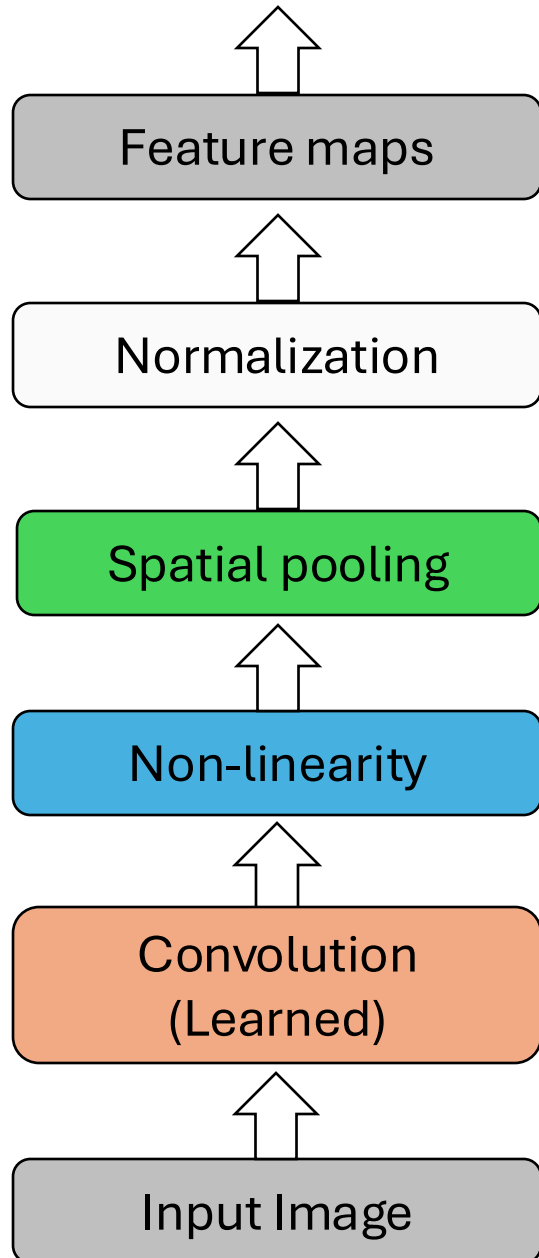


Input

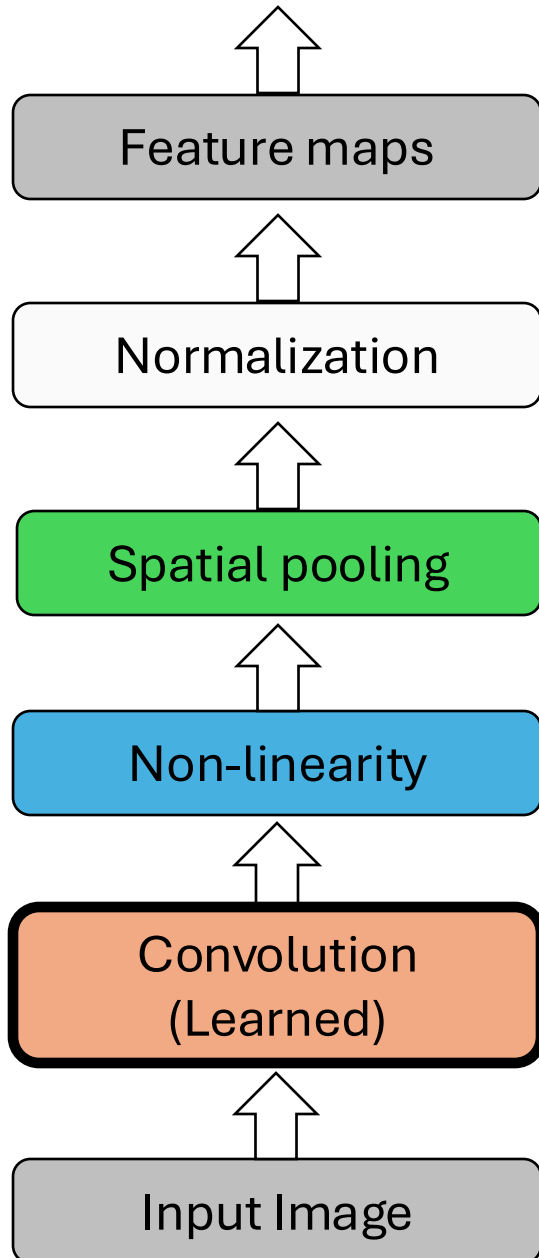


Feature Map

Convolutional Neural Networks



Convolutional Neural Networks

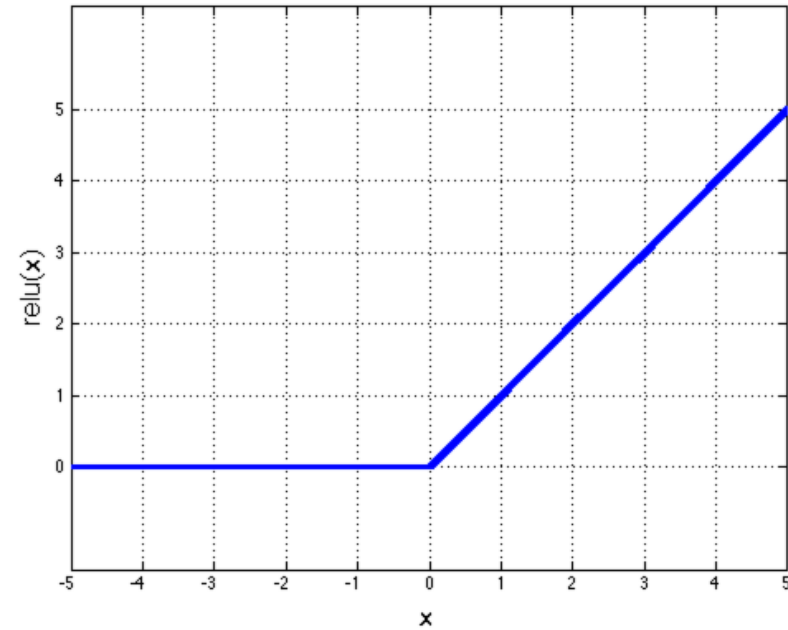
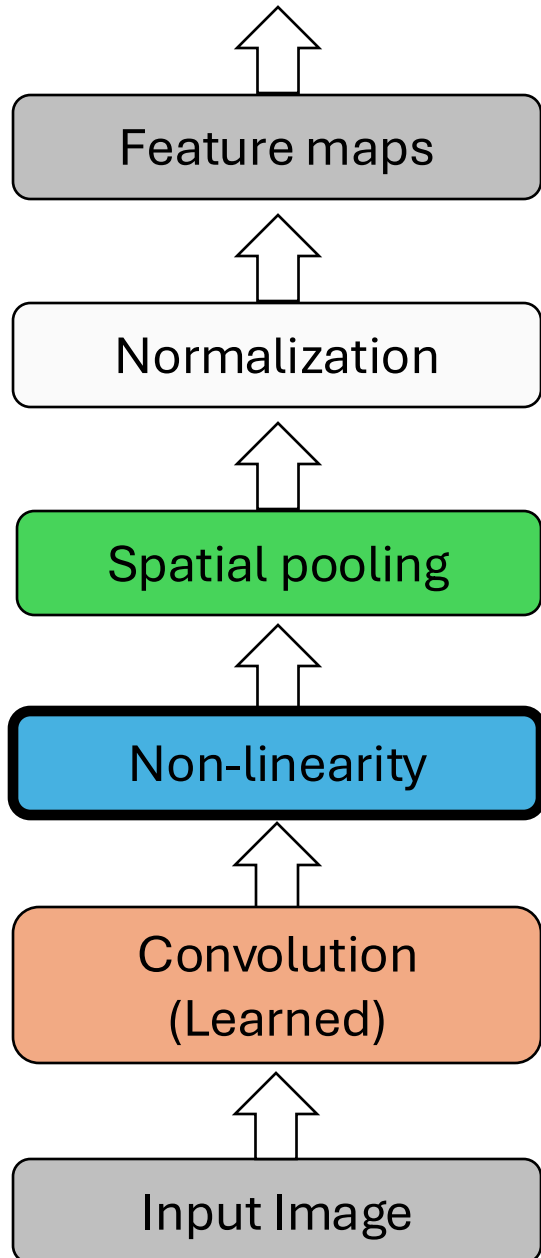


Input



Feature Map

Convolutional Neural Networks

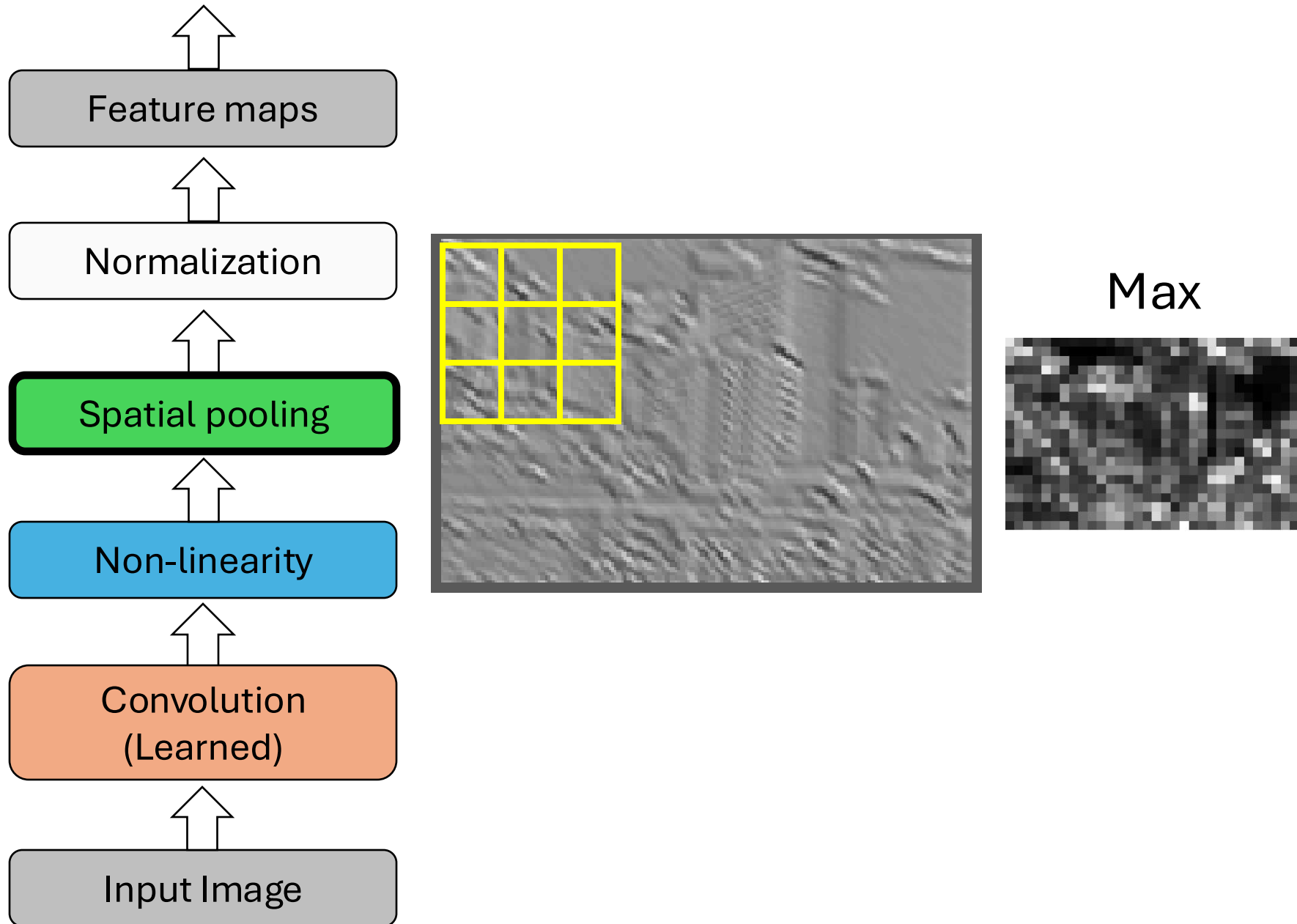


Rectified Linear Unit (ReLU)

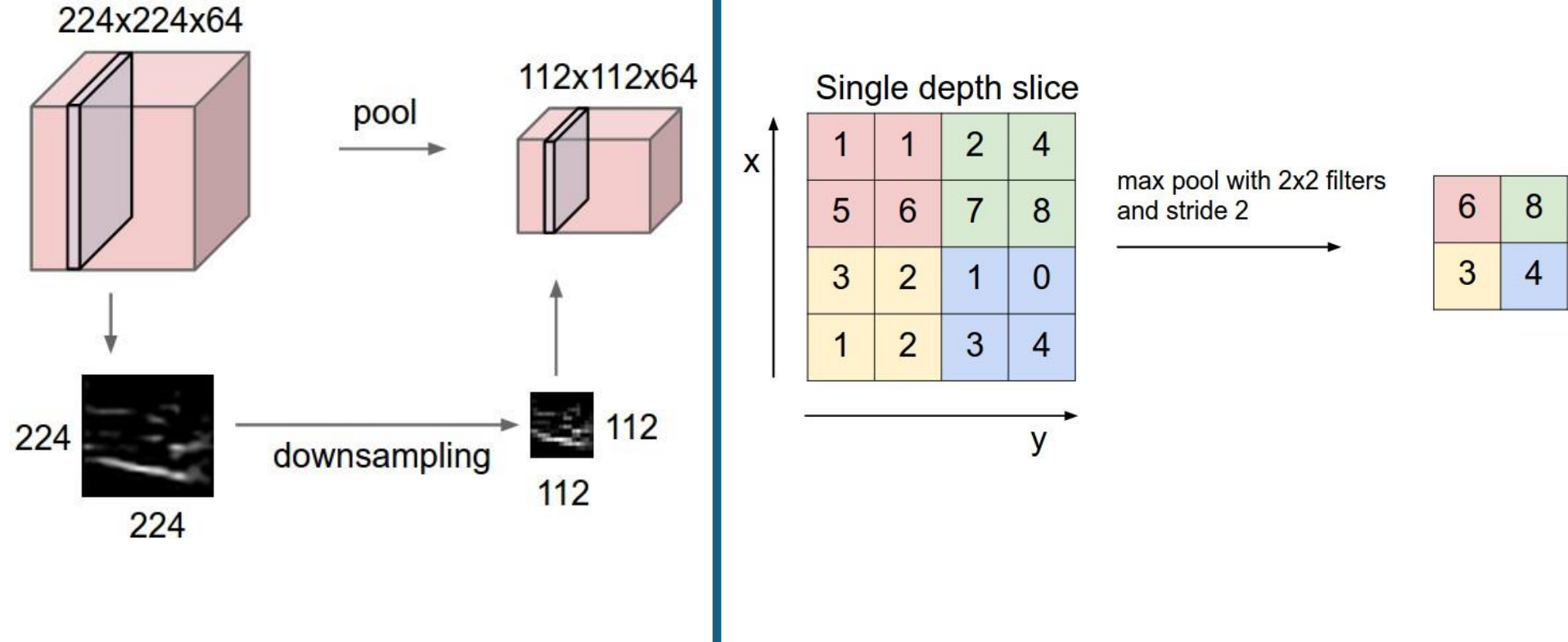
Exercise

- Rectified Linear Unit (ReLU): $y = \max(0, x)$
- If $x = -1$, what is y ?
- If $x = 0$, what is y ?
- If $y = 1$, what is x ?

Convolutional Neural Networks



Pooling

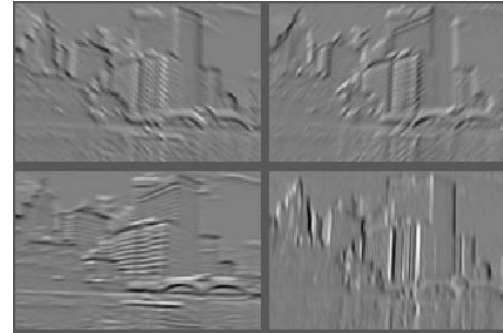
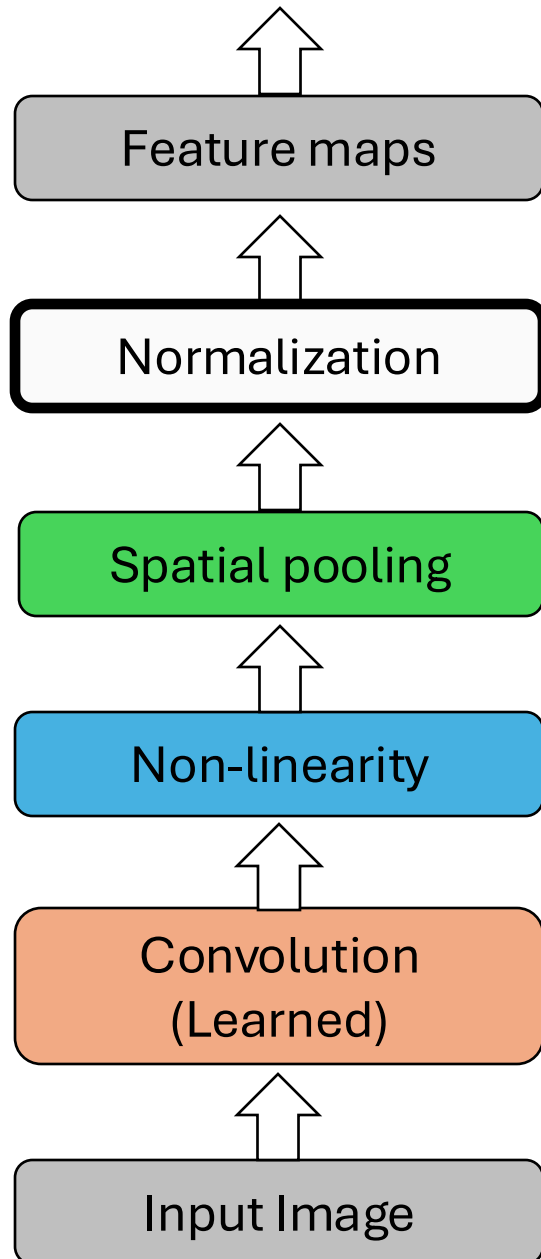


Input volume of size $[224 \times 224 \times 64]$ is pooled with filter size 2, stride 2 into output volume of size $[112 \times 112 \times 64]$.

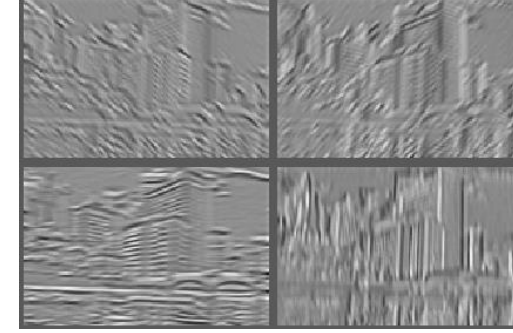
Exercise

- Max Pooling
- $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ Then what is the output?
- Average Pooling
- $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ Then what is the output?

Convolutional Neural Networks

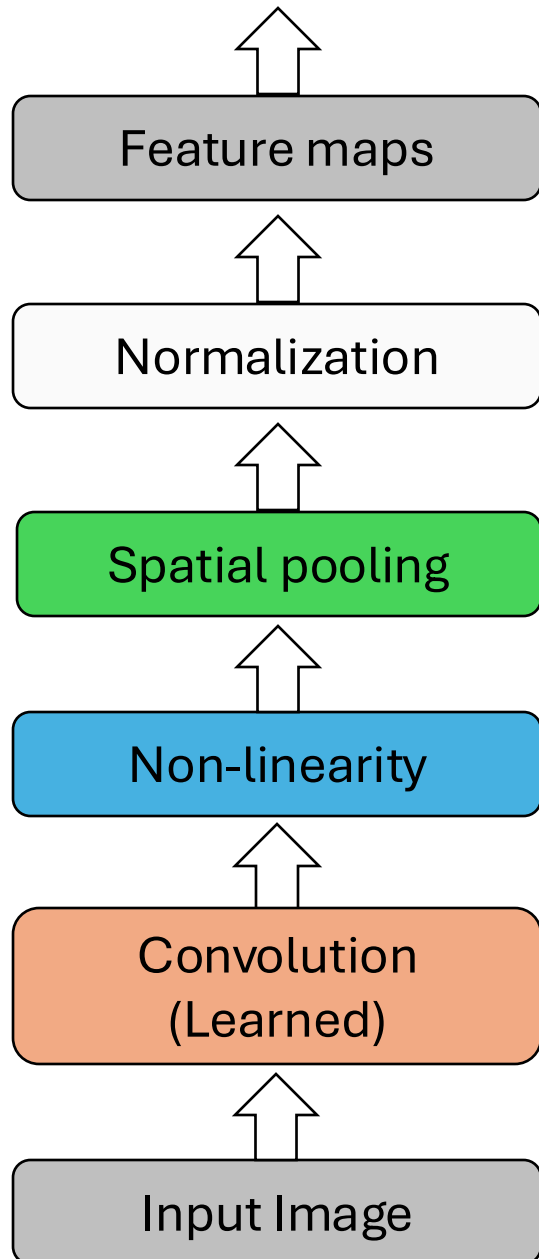


Feature Maps



Feature Maps
After Contrast
Normalization

Convolutional Neural Networks

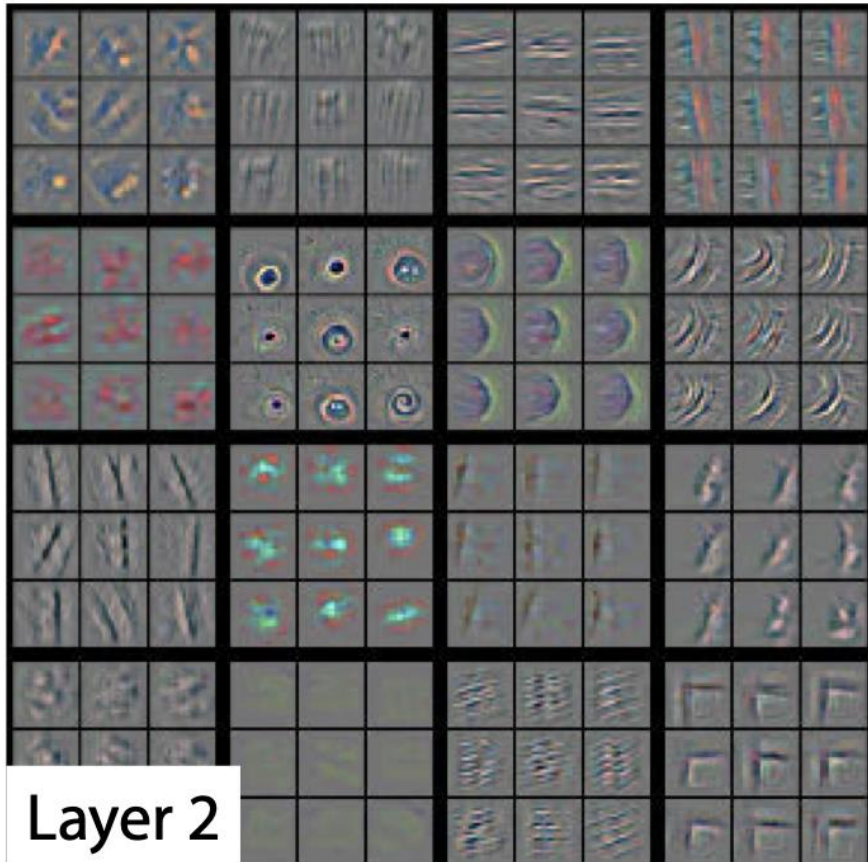


Convolutional filters are trained in a supervised manner by back-propagating classification error

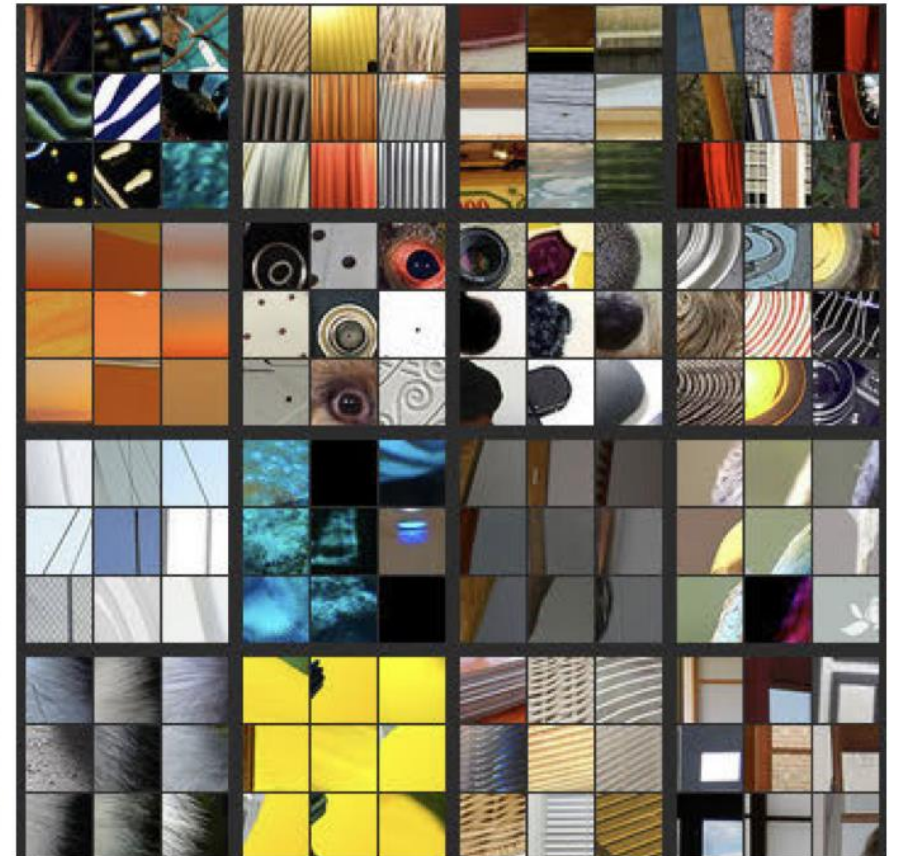
Visualization



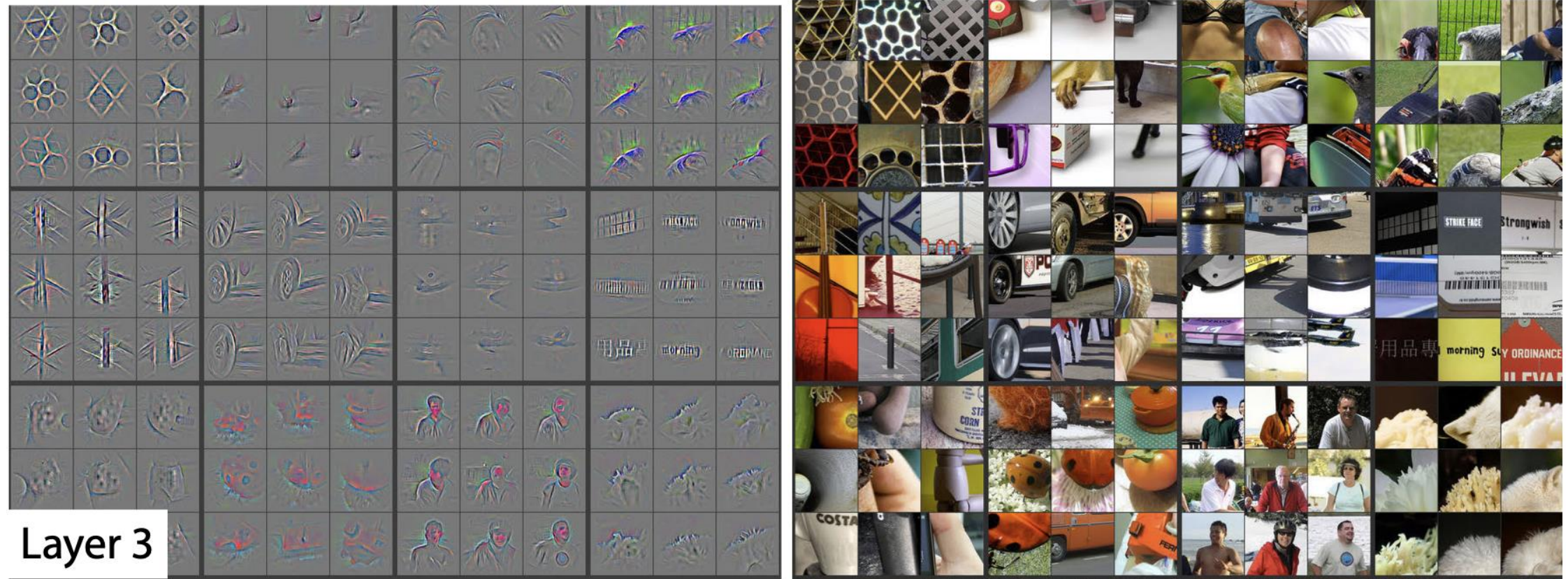
Layer 1



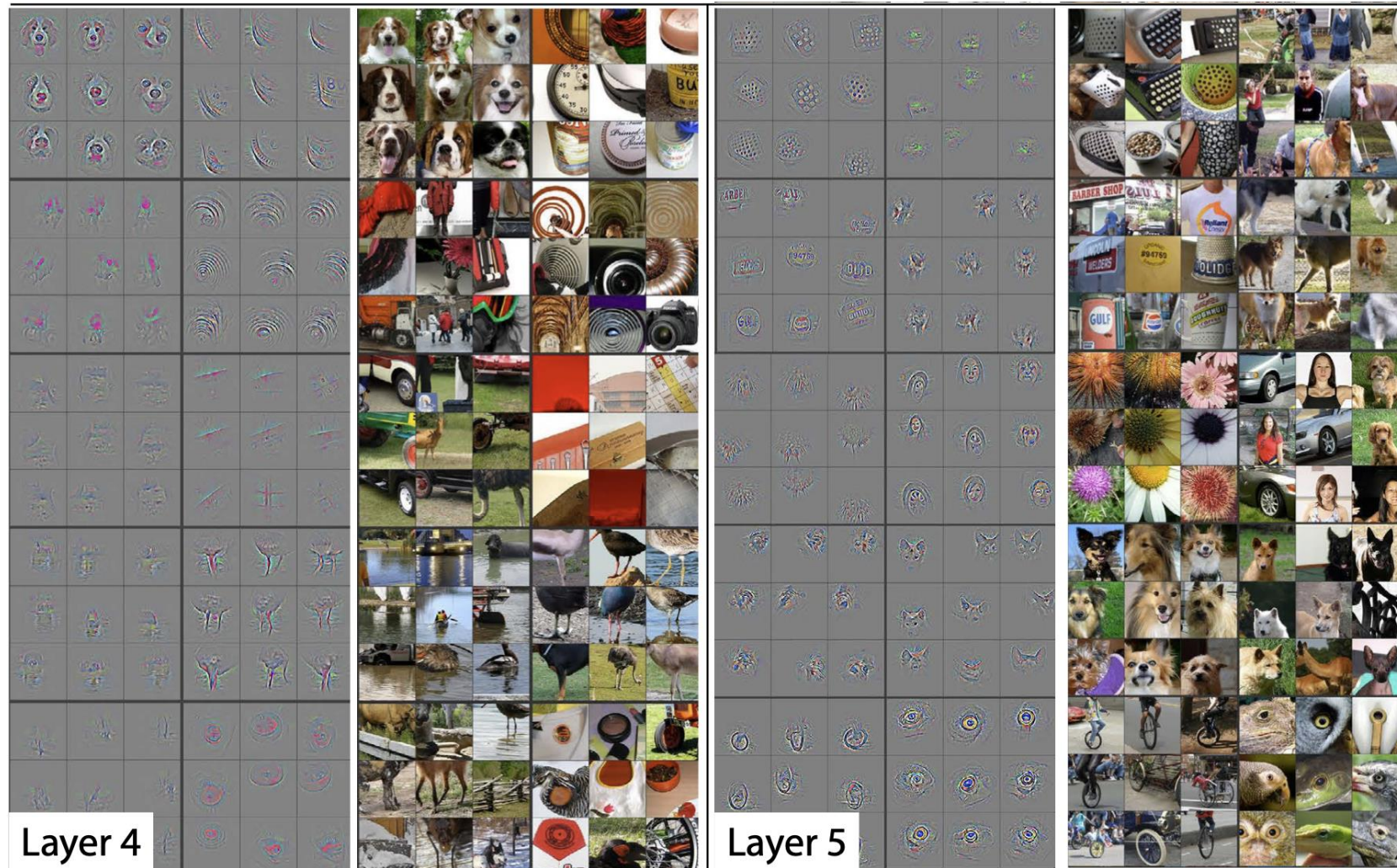
Layer 2



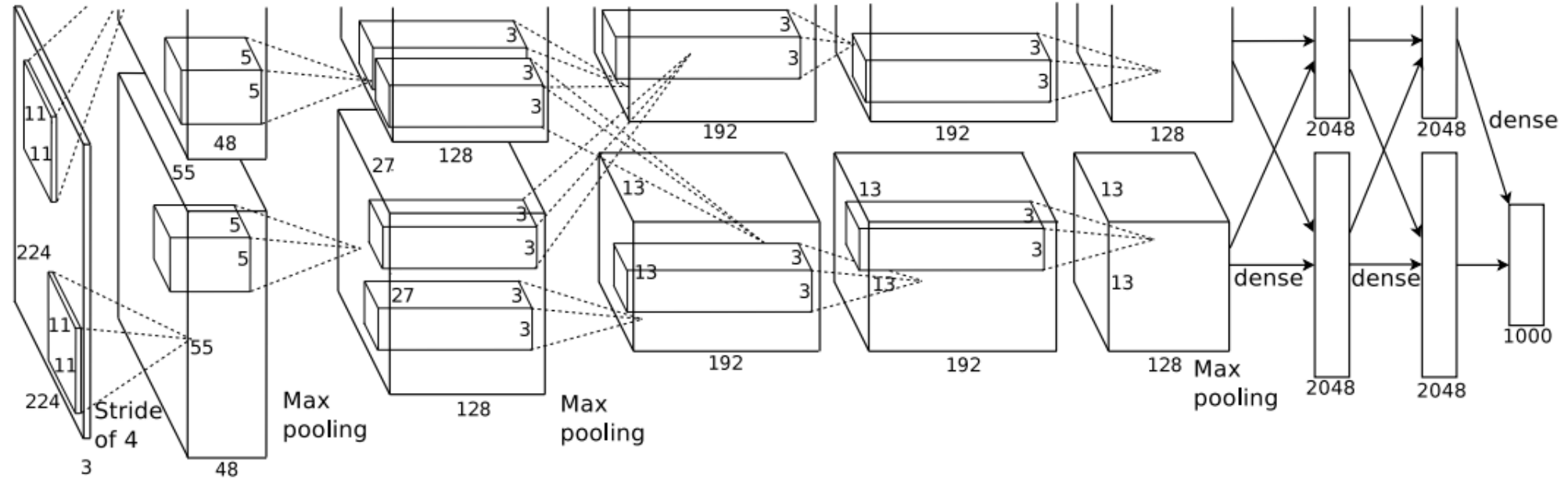
Visualization



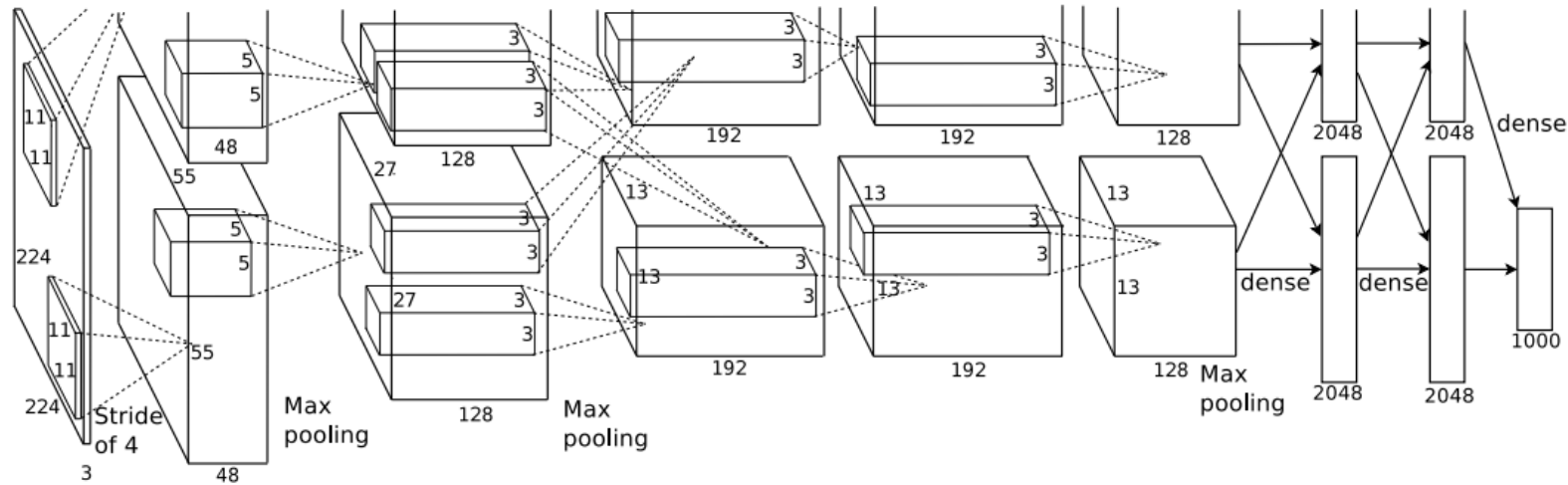
Visualization



First Breakthrough-AlexNet (2012)



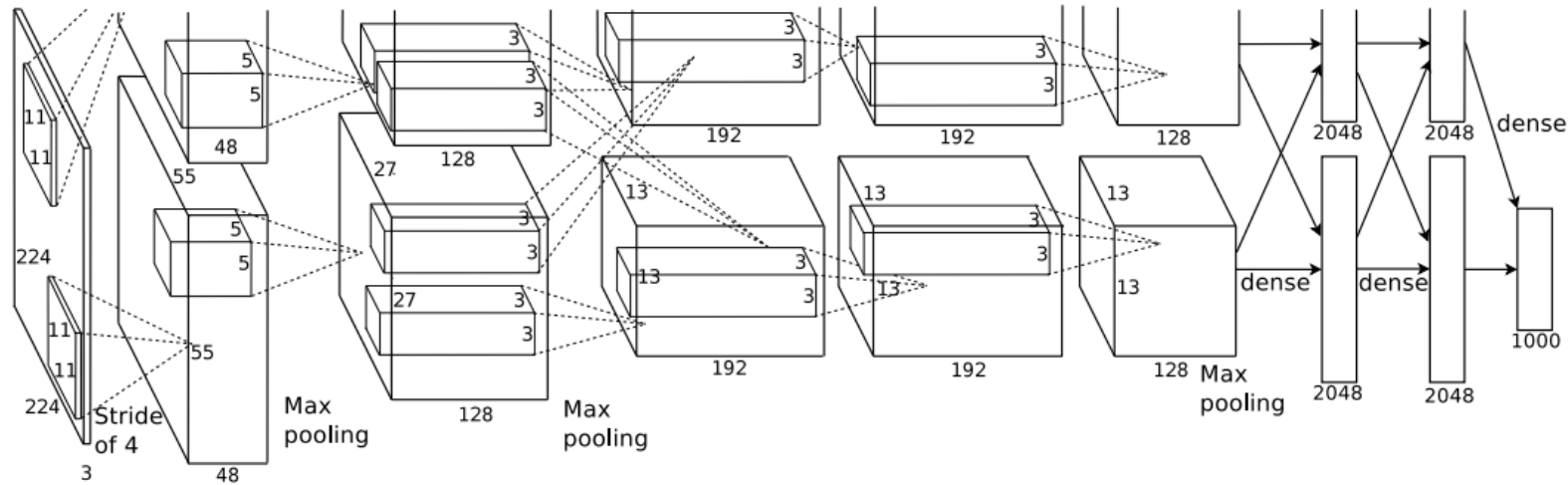
First Breakthrough-AlexNet (2012)



Input: 227*227*3

Conv1: 96 11*11*3 filters, with stride 4

First Breakthrough-AlexNet (2012)



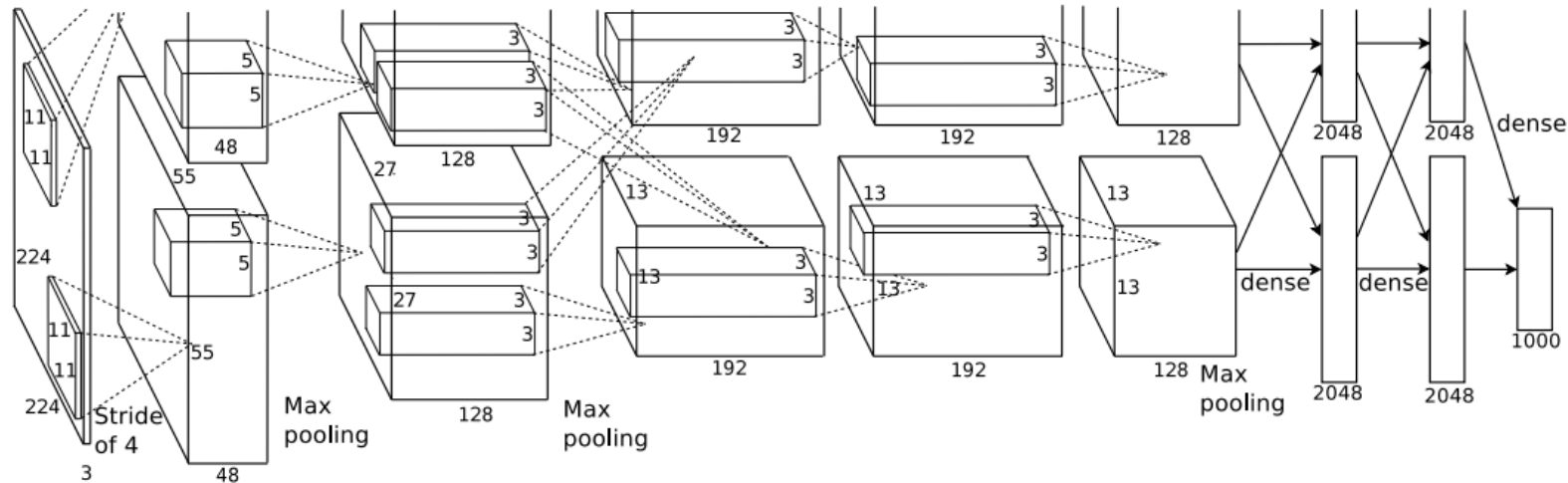
Input: $227 \times 227 \times 3$

Conv1: 96 $11 \times 11 \times 3$ filters, with stride 4

Output: $(227-11)/4+1=55$, $55 \times 55 \times 96$

??-????????

First Breakthrough-AlexNet (2012)



Input: $227 \times 227 \times 3$

Conv1: 96 $11 \times 11 \times 3$ filters, with stride 4

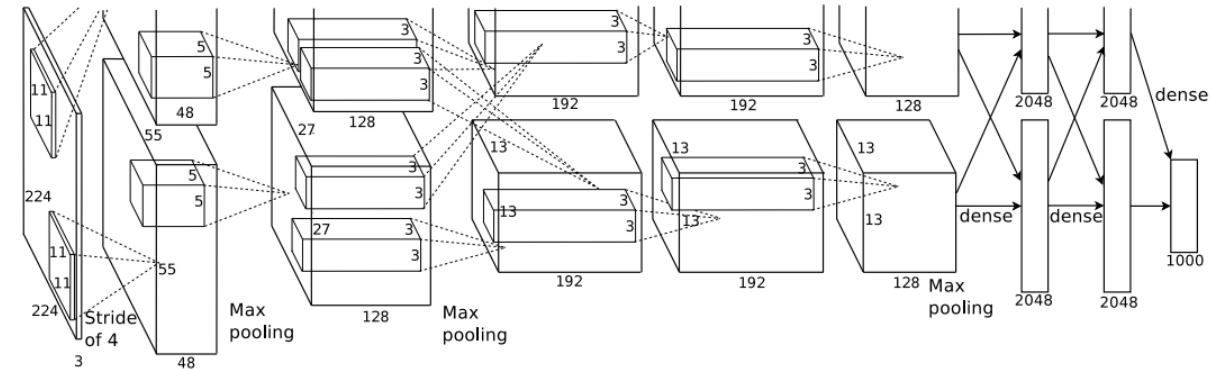
Output: $(227-11)/4+1=55$

Parameters: $11 \times 11 \times 3 \times 96 \approx 35K$

First Breakthrough-AlexNet (2012)

Details:

- first use of ReLU
- used Norm layers (not common anymore) - heavy data augmentation
- dropout 0.5
- batch size 128
- SGD Momentum 0.9
- Learning rate $1e-2$, reduced by 10 manually w accuracy plateaus
- L2 weight decay $5e-4$
- 7 CNN ensemble: 18.2% $\rightarrow$ 15.4%



AlexNet



96 convolutional kernels of size $11 \times 11 \times 3$ learned by the first convolutional layer on the $224 \times 224 \times 3$ input images.

AlexNet

```
class AlexNet(nn.Module):

    def __init__(self, num_classes=1000):
        super(AlexNet, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(64, 192, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(192, 384, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(384, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(256, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
        )
        self.avgpool = nn.AdaptiveAvgPool2d((6, 6))
        self.classifier = nn.Sequential(
            nn.Dropout(),
            nn.Linear(256 * 6 * 6, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(),
            nn.Linear(4096, 4096),
            nn.ReLU(inplace=True),
            nn.Linear(4096, num_classes),
        )
```

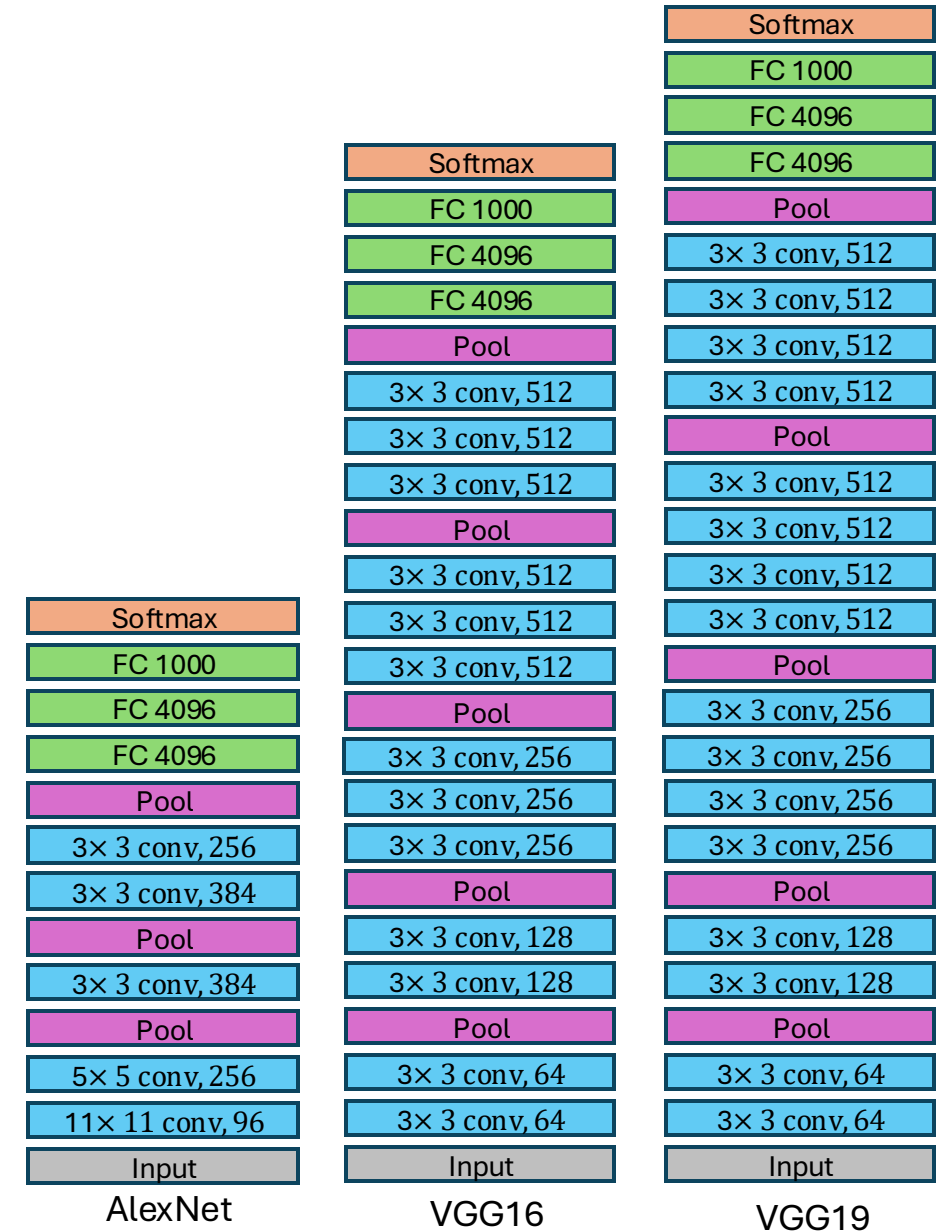
Feature Extractor

Classification

VGGNet

Small filters, Deeper networks

- 16 - 19 layers
- Only 3x3 CONV stride 1, pad 1 and 2x2 MAX POOL stride 2
- 16.4% top 5 error in ILSVRC'12 (AlexNet) -> 7.3% top 5 error in ILSVRC'14

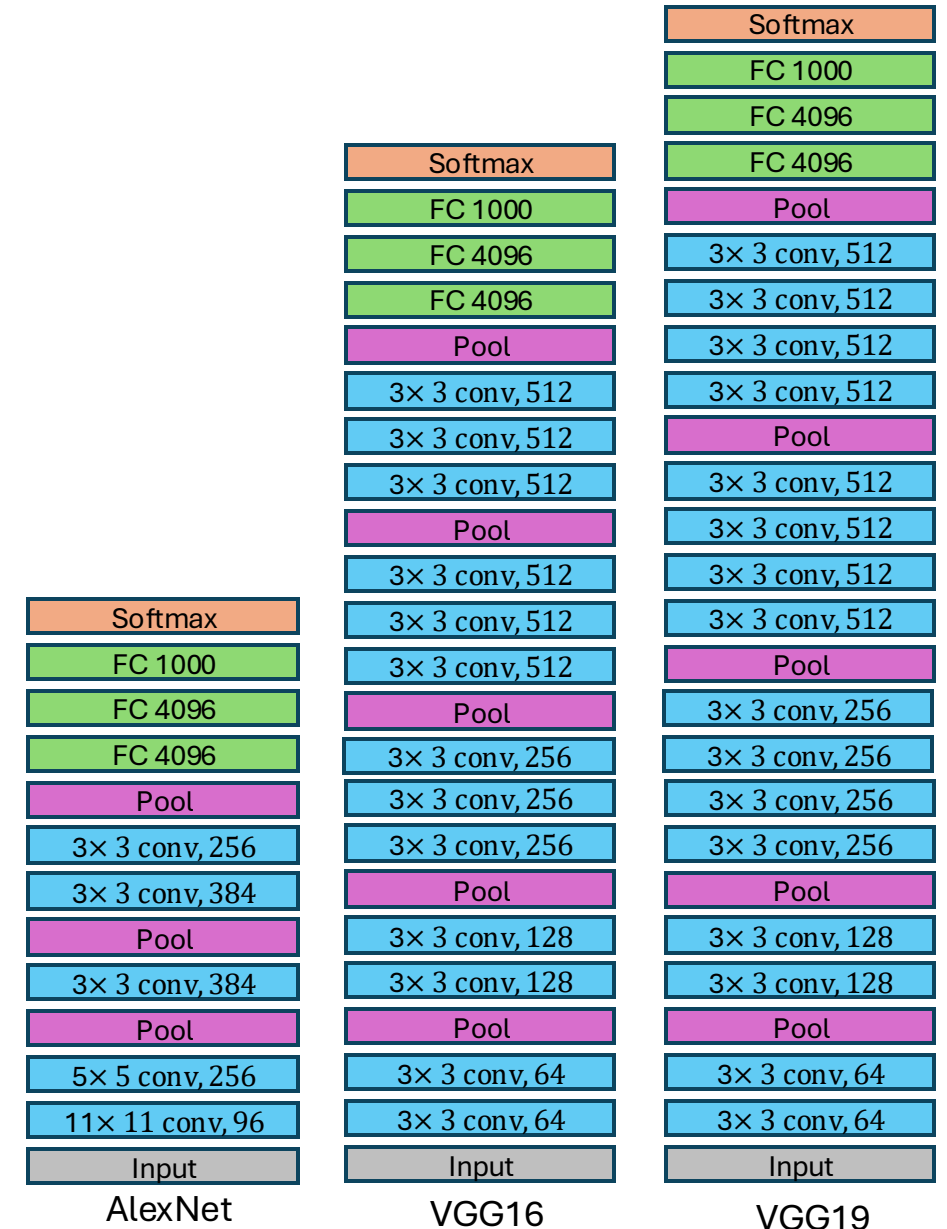


VGGNet

Why smaller filters?

Stack of two 3x3 conv (stride 1) layers has same **effective receptive field** as one 5x5 conv layer.

Stack of three 3x3 conv (stride 1) layers has same **effective receptive field** as one 7x7 conv layer.



VGGNet

INPUT: [224x224x3] memory: $224*224*3=150K$ params: 0 (not counting biases)

CONV3-64: [224x224x64] memory: $224*224*64=3.2M$ params: $(3*3*3)*64 = 1,728$

CONV3-64: [224x224x64] memory: $224*224*64=3.2M$ params: $(3*3*64)*64 = 36,864$

POOL2: [112x112x64] memory: $112*112*64=800K$ params: 0

CONV3-128: [112x112x128] memory: $112*112*128=1.6M$ params: $(3*3*64)*128 = 73,728$

CONV3-128: [112x112x128] memory: $112*112*128=1.6M$ params: $(3*3*128)*128 = 147,456$

POOL2: [56x56x128] memory: $56*56*128=400K$ params: 0

CONV3-256: [56x56x256] memory: $56*56*256=800K$ params: $(3*3*128)*256 = 294,912$

CONV3-256: [56x56x256] memory: $56*56*256=800K$ params: $(3*3*256)*256 = 589,824$

CONV3-256: [56x56x256] memory: $56*56*256=800K$ params: $(3*3*256)*256 = 589,824$

POOL2: [28x28x256] memory: $28*28*256=200K$ params: 0

CONV3-512: [28x28x512] memory: $28*28*512=400K$ params: $(3*3*256)*512 = 1,179,648$

CONV3-512: [28x28x512] memory: $28*28*512=400K$ params: $(3*3*512)*512 = 2,359,296$

CONV3-512: [28x28x512] memory: $28*28*512=400K$ params: $(3*3*512)*512 = 2,359,296$

POOL2: [14x14x512] memory: $14*14*512=100K$ params: 0

CONV3-512: [14x14x512] memory: $14*14*512=100K$ params: $(3*3*512)*512 = 2,359,296$

CONV3-512: [14x14x512] memory: $14*14*512=100K$ params: $(3*3*512)*512 = 2,359,296$

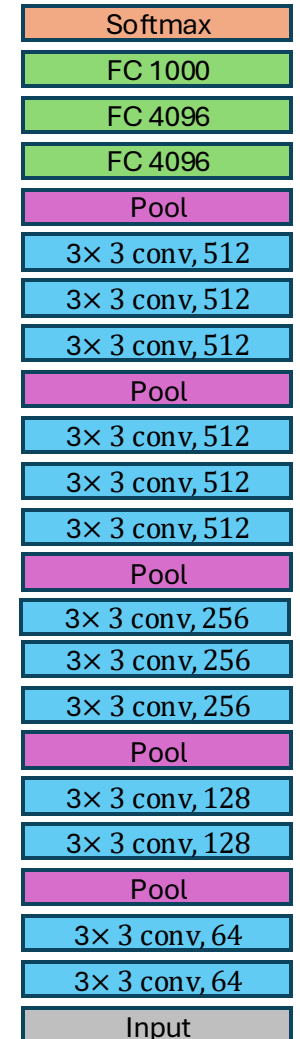
CONV3-512: [14x14x512] memory: $14*14*512=100K$ params: $(3*3*512)*512 = 2,359,296$

POOL2: [7x7x512] memory: $7*7*512=25K$ params: 0

FC: [1x1x4096] memory: 4096 params: $7*7*512*4096 = 102,760,448$

FC: [1x1x4096] memory: 4096 params: $4096*4096 = 16,777,216$

FC: [1x1x1000] memory: 1000 params: $4096*1000 = 4,096,000$

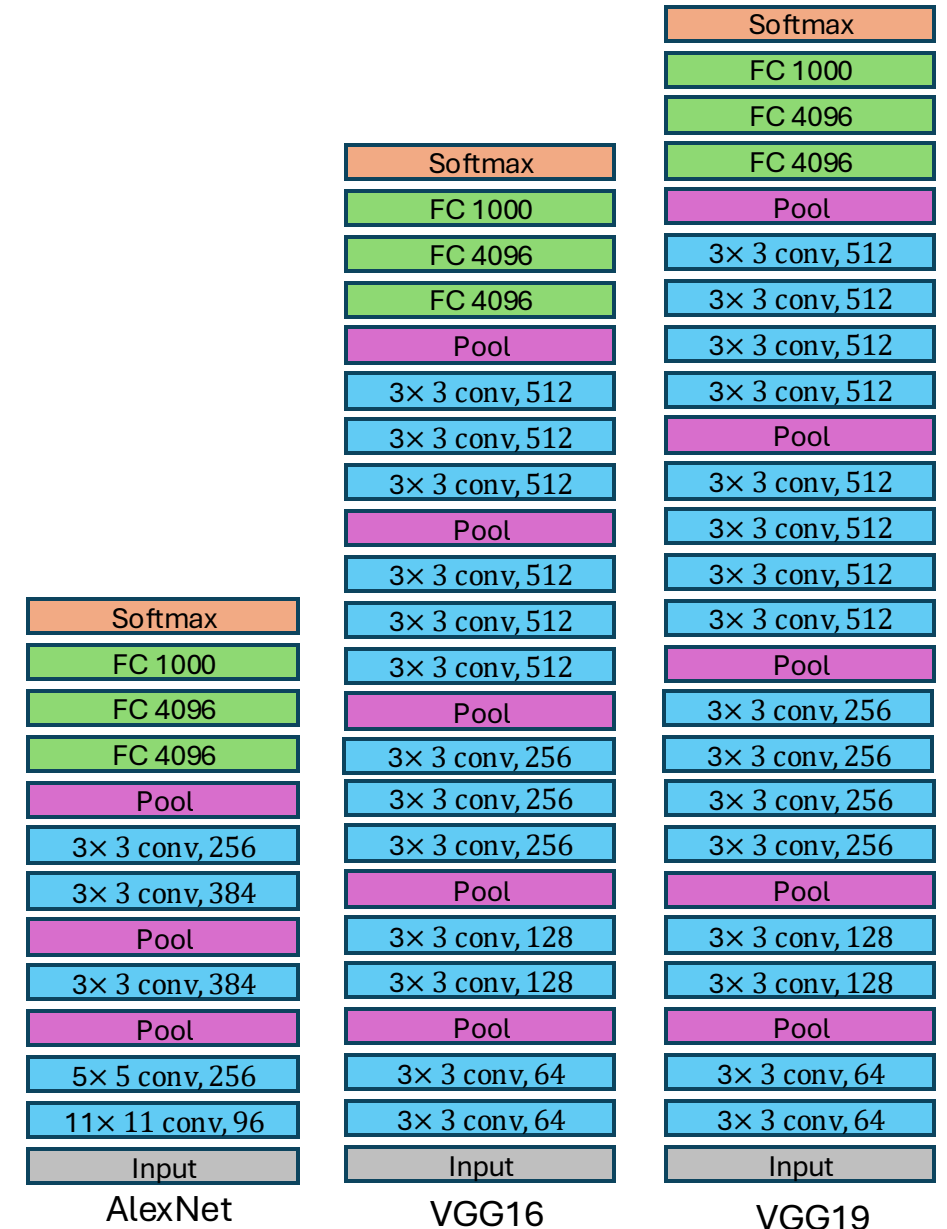


VGG16

VGGNet

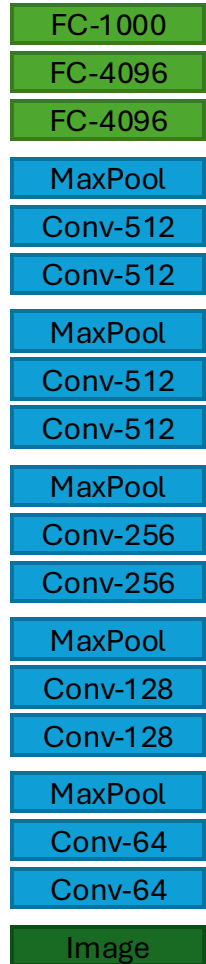
Details:

- ILSVRC'14 2nd in classification, 1st in localization
- Similar training procedure as Krizhevsky 2012
- No Local Response Normalisation (LRN)
- Use VGG16 or VGG19 (VGG19 only slightly better, more memory)
- Use ensembles for best results
- FC7 features generalize well to other tasks

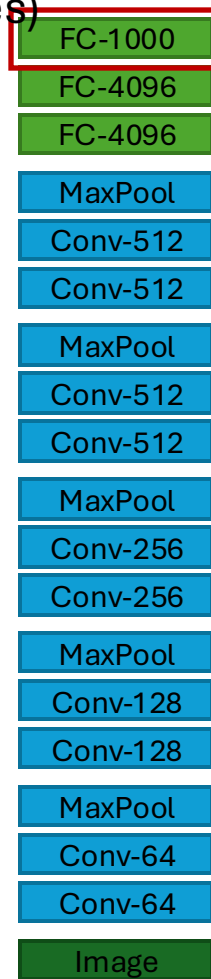


How to Train CNNs?

1. Train on ImageNet



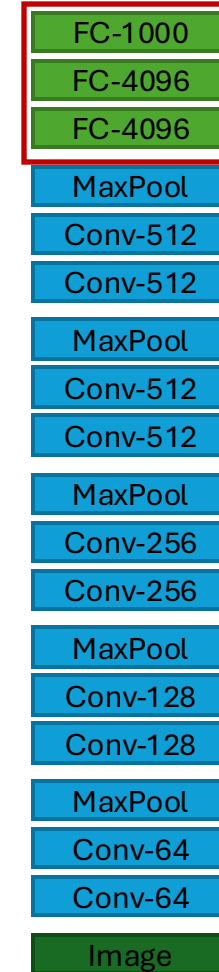
2. Small Dataset (C classes)



Reinitialize
This and train

Freeze these

3. Bigger dataset



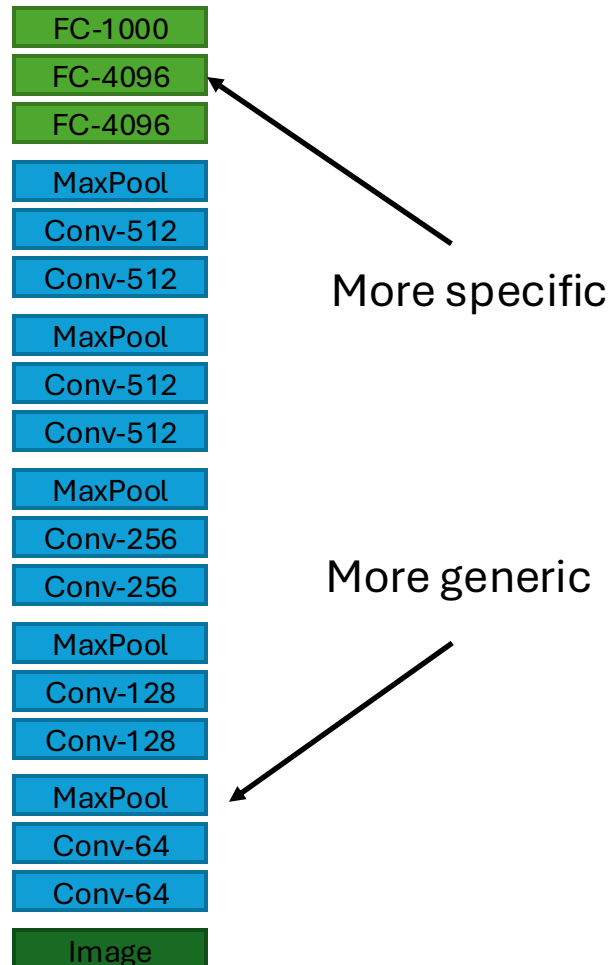
Train these

With bigger
dataset, train more
layers

Freeze these

Lower learning rate
when finetuning;
1/10 of original LR
is good starting
point

How to Train CNNs?



	Very similar dataset	Very different dataset
Very little data	Use linear classifier on top layer	Not good! Try linear classifier
Quite a lot of data	Finetune a few layers	Finetune a large number of layers

How to Train CNNs?

```
import torchvision.models as models
resnet18 = models.resnet18(pretrained=True)
alexnet = models.alexnet(pretrained=True)
squeezenet = models.squeezenet1_0(pretrained=True)
vgg16 = models.vgg16(pretrained=True)
densenet = models.densenet161(pretrained=True)
inception = models.inception_v3(pretrained=True)
googlenet = models.googlenet(pretrained=True)
shufflenet = models.shufflenet_v2_x1_0(pretrained=True)
mobilenet = models.mobilenet_v2(pretrained=True)
resnext50_32x4d = models.resnext50_32x4d(pretrained=True)
wide_resnet50_2 = models.wide_resnet50_2(pretrained=True)
mnasnet = models.mnasnet1_0(pretrained=True)
```

TORCHVISION.MODELS

How to Train CNNs?

Two major transfer learning scenarios:

- **Finetuning the convnet:** Instead of random initialization, we initialize the network with a pretrained network, like the one that is trained on imagenet 1000 dataset. Rest of the training looks as usual.
- **ConvNet as fixed feature extractor:** Here, we will freeze the weights for all of the network except that of the final fully connected layer. This last fully connected layer is replaced with a new one with random weights and only this layer is trained.

How to Train CNNs?

```
model_ft = models.resnet18(pretrained=True)
```

Model
definition

```
num_fts = model_ft.fc.in_features
```

```
# Here the size of each output sample is set to 2.
```

```
# Alternatively, it can be generalized to nn.Linear(num_fts, len(class_names)).
```

```
model_ft.fc = nn.Linear(num_fts, 2)
```

Model
finetuned

```
model_ft = model_ft.to(device)
```

```
criterion = nn.CrossEntropyLoss()
```

Loss function

```
# Observe that all parameters are being optimized
```

```
optimizer_ft = optim.SGD(model_ft.parameters(), lr=0.001, momentum=0.9)
```

Optimizer

```
# Decay LR by a factor of 0.1 every 7 epochs
```

```
exp_lr_scheduler = lr_scheduler.StepLR(optimizer_ft, step_size=7, gamma=0.1)
```

Learning rate
policy

```
model_ft = train_model(model_ft, criterion, optimizer_ft, exp_lr_scheduler,  
                        num_epochs=25)
```

Model Training

How to Train CNNs?

```
model_conv = torchvision.models.resnet18(pretrained=True)
for param in model_conv.parameters():
    param.requires_grad = False  # freeze the parameters

# Parameters of newly constructed modules have requires_grad=True by default
num_ftrs = model_conv.fc.in_features
model_conv.fc = nn.Linear(num_ftrs, 2)

model_conv = model_conv.to(device)

criterion = nn.CrossEntropyLoss()

# Observe that only parameters of final layer are being optimized as
# opposed to before.
optimizer_conv = optim.SGD(model_conv.fc.parameters(), lr=0.001, momentum=0.9)

# Decay LR by a factor of 0.1 every 7 epochs
exp_lr_scheduler = lr_scheduler.StepLR(optimizer_conv, step_size=7, gamma=0.1)
```

MTCNN

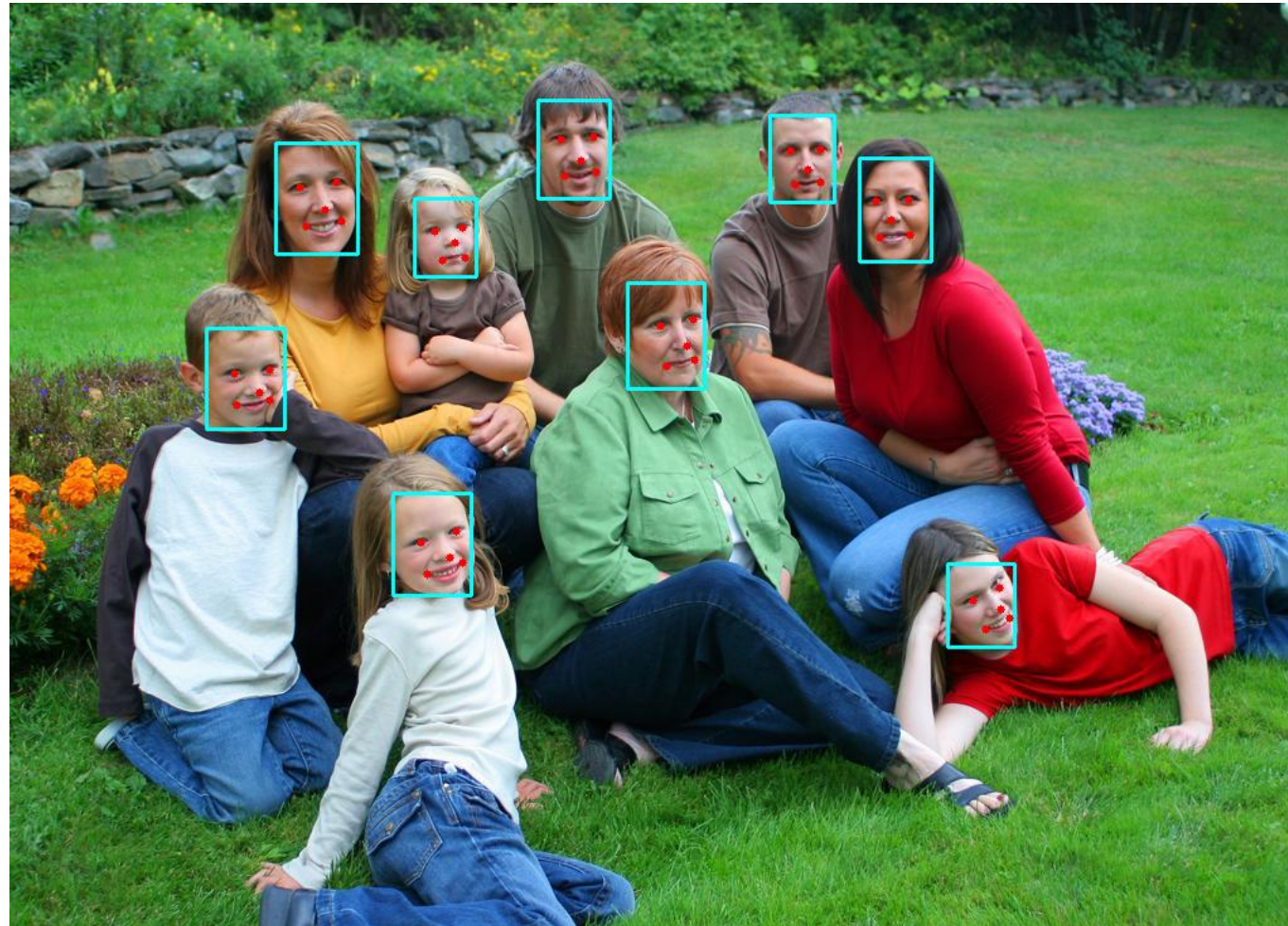


Image is from WiderFace dataset